

WARP: An Efficient Engine for Multi-Vector Retrieval

Jan Luca Scheerer*
ETH Zurich
Zurich, Switzerland
lscheerer@ethz.ch

Matei Zaharia
UC Berkeley
Berkeley, CA, USA
matei@berkeley.edu

Christopher Potts
Stanford University
Stanford, CA, USA
cgpotts@stanford.edu

Gustavo Alonso
ETH Zurich
Zurich, Switzerland
alonso@inf.ethz.ch

Omar Khattab
Stanford University
Stanford, CA, USA
okhattab@cs.stanford.edu

Abstract

Multi-vector retrieval methods such as ColBERT and its recent variant, the ConteXtualized Token Retriever (XTR), offer high accuracy but face efficiency challenges at scale. To address this, we present WARP, a retrieval engine that substantially improves the efficiency of retrievers trained with the XTR objective through three key innovations: (1) WARP_{SELECT} for dynamic similarity imputation; (2) implicit decompression, avoiding costly vector reconstruction during retrieval; and (3) a two-stage reduction process for efficient score aggregation. Combined with highly-optimized C++ kernels, our system reduces end-to-end latency compared to XTR’s reference implementation by 41x, and achieves a 3x speedup over the ColBERTv2/PLAID engine, while preserving retrieval quality.

 <https://github.com/jlscheerer/xtr-warp>

Keywords

Dense Retrieval, Multi-Vector, Late Interaction, Efficiency

ACM Reference Format:

Jan Luca Scheerer, Matei Zaharia, Christopher Potts, Gustavo Alonso, and Omar Khattab. 2025. WARP: An Efficient Engine for Multi-Vector Retrieval. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR ’25)*, July 13–18, 2025, Padua, Italy. Preprint, 9 pages. <https://doi.org/10.1145/3726302.3729904>

1 Introduction

Over the past several years, information retrieval (IR) research has introduced new neural paradigms for search based on pretrained Transformers. Central among these, the late interaction paradigm proposed in ColBERT [8] departs from the bottlenecks of conventional *single-vector* representations. Instead, it encodes queries and documents into *multi-vector* representations on top of which it is able to scale gracefully to search massive collections.

*Work completed as a visiting student researcher at Stanford University.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGIR ’25, July 13–18, 2025, Padua, Italy

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1592-1/2025/07

<https://doi.org/10.1145/3726302.3729904>

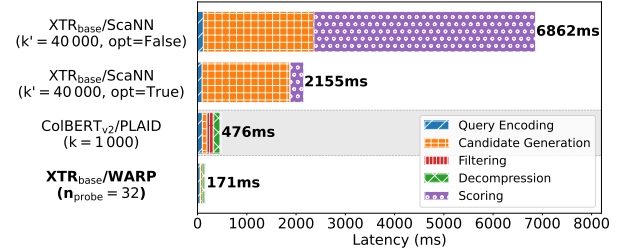


Figure 1: Single-threaded CPU latency breakdown of (1) XTR’s unoptimized reference implementation, (2) a variant of XTR that we optimized, (3) the official ColBERTv2/PLAID system, and (4) our proposed XTR/WARP on LoTTE Pooled.

Since the original ColBERT architecture was introduced, there has been substantial research in optimizing the latency of multi-vector retrieval models [2, 13, 15]. Most notably, PLAID [18] reduces late interaction search latency by 45x on a CPU compared to a vanilla ColBERTv2 [19] process, while continuing to deliver state-of-the-art retrieval quality. Orthogonally, the ConteXtualized Token Retriever (XTR) [11] introduces a novel training objective that eliminates the need for a separate gathering stage and thereby significantly simplifies the subsequent scoring stage. While XTR¹ lays extremely promising groundwork for more efficient multi-vector retrieval, we find that it naively relies on a general-purpose vector similarity search library (ScaNN) and combines that with unoptimized Python data structures and manual iteration, introducing substantial overhead.

The key insights from PLAID and XTR appear rather isolated. Whereas PLAID is concerned with aggressively and swiftly pruning away documents it finds unpromising, XTR tries to eliminate gathering complete document representations in the first place. We ask whether there are potentially rich interactions between these two fundamentally distinct approaches to speeding up multi-vector search. To study this, we introduce a new engine for retrieval with XTR-based ColBERT models, called WARP, that combines techniques from ColBERTv2/PLAID with innovations tailored for the XTR architecture. Our contributions in WARP include: (1) the WARP_{SELECT} method for imputing missing similarities, (2) a new method for implicit decompression of vectors during search, and (3) a novel two-stage reduction phase for efficient scoring.

¹<https://github.com/google-deepmind/xtr>